

Research article

CEPEDALoCo: An event-driven architecture for integrating complex event processing and blockchain through low-code

Jesús Rosa-Bilbao^{a,*}, Juan Boubeta-Puig^{a,*}, Adrian Rutle^b

^a UCASE Software Engineering Research Group, Department of Computer Science and Engineering, University of Cadiz, Avda. de la Universidad de Cádiz 10, 11519 Puerto Real, Cádiz, Spain

^b Department of Computer Science, Electrical Engineering and Mathematical Sciences, Western Norway University of Applied Sciences, Bergen, Norway

ARTICLE INFO

Dataset link: <https://doi.org/10.17632/ngshc6y5kv.1>

Keywords:

Air quality
Blockchain
Complex event processing
Event-driven architecture
Internet of Things
Low-code

ABSTRACT

Internet of Things (IoT) is made up of millions of devices generating large amounts of heterogeneous data from multiple sources. These devices can be from multiple manufacturers which makes their use in terms of data acquisition, processing, analysis and actions on these data challenging. Additionally, all these data must be analyzed and correlated in real time for the early detection of situations of interest (complex events) and subsequent decision making. These complex events must be able to automatically trigger decisions and be stored in a secure, immutable and accessible way. In this context, Event-Driven Applications (EDAs) are a solution to meet these needs, however, developing such applications requires vast knowledge in certain technologies. To address these challenges, an EDA is proposed in this paper to integrate Complex Event Processing (CEP) and blockchain through the low-code paradigm. This proposal allows for the development of EDAs in a user-friendly way. These applications make it possible to integrate IoT devices from multiple manufacturers and with different data formats together with CEP technology for complex event detection and blockchain for secure, immutable and accessible event storage. To demonstrate the feasibility, the architecture was applied and evaluated in a case study related to measuring and acting on air quality using IoT devices that measure different pollutants and factors such as temperature, humidity and wind. The results show that the graphically designed EDAs facilitate the real-time analysis of the collected IoT data via a CEP engine, whose outcome is transparently and automatically registered in a blockchain network.

1. Introduction

In the last few years, the number of Internet of Things (IoT) devices available in the market has increased considerably [1]. Specifically, 7 billion devices were registered in 2018 and it is expected to reach 21.5 billion by 2025 [2]. These devices are typically marketed by different manufacturers and have different ways of providing their data [3]. Despite the benefits and opportunities which come with the abundance of IoT devices, there are multiple related challenges which have to be addressed in order to fully unfold their potentials [4].

The problem definition can be described in terms of the following challenges. Firstly, IoT devices from different manufacturers use different data acquisition techniques and formats, making the alignment and analysis of the data challenging [3]. Secondly,

* Corresponding author.

E-mail address: juan.boubeta@uca.es (J. Boubeta-Puig).

<https://doi.org/10.1016/j.iot.2023.100802>

Received 17 February 2023; Received in revised form 21 April 2023; Accepted 27 April 2023

Available online 2 May 2023

2542-6605/© 2023 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

the real-time analysis and correlation of huge amounts of heterogeneous data generated by millions of IoT devices with the aim of early detecting situations of interest (a.k.a. complex events) for the domain in question and then automatically making decisions accordingly is also challenging [5]. Finally, the notification of these complex events to third-party systems and their storage in a secure, decentralized, immutable, accessible and traceable way is another of the current challenges in IoT application domains [6].

Event-Driven Applications (EDAs) are a possible solution to meet these needs, however, the development of such applications requires vast knowledge in certain technologies such as Complex Event Processing (CEP) and blockchain [7]. On the one hand, CEP technology enables automatic detection of complex events and real-time decision making by matching previously defined event patterns [8]. Nevertheless, the implementation of these event patterns usually requires high skills in the Event Processing Languages (EPLs) supported by this technology [9]. On the other hand, blockchain technology is a distributed ledger that can store data ensuring its security, integrity, traceability, immutability and transparency due to its decentralized nature [10], but its use also requires a high degree of expertise.

In a previous work, a low-code approach, called EDALoCo [6], was proposed for the development of EDAs for smart contract management. Specifically, it aimed to improve the accessibility of blockchain technology by facilitating operations with smart contracts, such as compiling, deploying, interacting or monitoring them. In addition, the proposal was applied to a case study based on COVID-19 vaccines with the use of IoT sensors. However, real-time event analysis and correlation technology was not used to obtain meaningful information that would allow to take actions based on a set of defined rules, such as the automatic invocation of smart contracts.

To address the mentioned challenges on IoT, CEP and blockchain, and bridge the knowledge gap between these cutting-edge technologies, this paper aims to propose an EDA, called CEPEDALoCo, for integrating IoT [11], CEP and blockchain technologies through the low-code paradigm. This integration allows for the creation of EDAs in a user-friendly way. Through these applications, software developers, who are not experts in CEP and blockchain, will be able to analyze large amounts of data in real time from multiple data sources and IoT devices regardless of their manufacturer or data format in order to early detect complex events, which will have the ability to trigger actions for being automatically stored in a blockchain network in a secure, immutable and accessible manner. Thanks to the use of the low-code paradigm [12], the development of this type of applications allows generating value faster, greater flexibility, automating tasks, reusing components and integrating these applications with other systems.

The proposed architecture is composed of 3 layers. The *data production* layer, in which large amounts of data are generated by IoT devices, Application Programming Interfaces (APIs), blockchain networks, etc. The *data processing* layer, which is the main contribution of this paper where the integration of IoT, CEP and blockchain takes place. Finally, the *data consumption* layer that is in charge of receiving the complex events produced in the data processing layer, being stored in a blockchain network and monitored through dashboards or social networks.

To make this integration, the Node-RED low-code open-source platform [13] has been extended by developing new nodes that allow for the connection and use of blockchain networks within the platform. In addition, data exchange with a CEP platform has been conducted through several components, such as message brokers and APIs.

This architecture can be applied to multiple IoT domains. To demonstrate its feasibility, the architecture has been applied to a real-world case study on air quality. Particularly, this case study conducts the real-time analysis and processing of data from IoT sensors deployed in different locations in Spain. This processing may give rise to situations of interest, such as a CO2 level dangerous to health or an inadequate humidity level that facilitates the appearance of bacteria. These situations of interest produced in real time can be automatically stored in a secure, transparent, immutable and traceable way in a blockchain network through the EDAs graphically modeled via the low-code platform. Additionally, these situations of interest can be notified to users in real time via social networks, such as Telegram. Furthermore, this architecture has been tested by a group of independent developers who are not experts in the technologies used in this work, what has allowed to evaluate some aspects such as the functionality.

The contributions of this paper, which have not been addressed altogether by existing architectures, are summarized as follows:

- **C1:** Facilitating the integration of multiple IoT devices regardless of data format and manufacturer.
- **C2:** Analyzing and correlating real-time streams of data to early detect complex events and reacting quickly to them by triggering a series of actions.
- **C3:** Storing complex events ensuring security, immutability, transparency, decentralization and traceability.
- **C4:** Developing EDAs integrating IoT, CEP and blockchain in a user-friendly way through low-code paradigm.

The rest of the paper is organized as follows. Section 2 describes the technologies and paradigms used in this work. Section 3 compares the work presented with other ones. Section 4 presents the novel architecture for the integration of CEP and blockchain through a low-code approach. Section 5 describes the real-world case study on air quality in which the proposal has been applied, and its results and evaluation are discussed in Section 6. Finally, Section 7 shows the conclusions and outlines future work.

2. Background

In this section, the background on the technologies and paradigms used in the proposal is described: CEP, blockchain and low-code.

2.1. CEP

CEP is a cutting-edge technology that allows processing real-time data continuously to identify significant events or combinations of events based on a predefined set of rules [9].

An event is defined as anything that occurs, such as a change of state. CEP is the method of processing multiple event streams and correlating seemingly unrelated events to identify opportunities or threats. An event occurrence or event sequence requiring an immediate reaction is known as a *situation of interest* [14].

When the conditions previously defined in an event pattern are satisfied a complex event is automatically created by a *CEP engine*. This is a powerful software that provides programmers with the ability to implement such patterns by using EPLs [15]. Additionally, a CEP engine can trigger real-time actions and alerts upon pattern detection.

CEP technology has several benefits, such as faster and automatic reply, human workload reduction, information overload prevention and decision quality improvement. Thereby, CEP-based systems have gained many applications in a variety of areas including smart cities [16], logistics [7], IoT [17], monitoring critical infrastructure and financial applications [18].

In this work, CEP is used in the proposed CEPEDALoCo architecture for the detection of complex events in diverse IoT application domains, such as unhealthy AQI levels or inadequate temperatures in an environment area.

2.2. Blockchain

Blockchain can be defined as a structure for storing data in a way that is difficult to falsify. It is a public electronic ledger that can be shared openly among users and creates an immutable record of their transactions. All transactions must be verified, recorded and combined with other transactions to create new blocks [19]. It is having a considerable impact on areas such as e-health [20,21], smart industry [22], cybersecurity [23], smart cities [24], IoT [25], education [26] and voting [27].

Blockchain can only be updated by consensus among the participants in the network, and when new data is introduced, it can never be deleted. There is a true and verifiable record of each and every entry made into the blockchain network [28].

Nowadays there are many blockchain platforms, however, some of the most known ones belong to the Ethereum Virtual Machine (EVM) [29]. EVM is a piece of software that executes smart contracts and calculates the state of the Ethereum network after each new block is added to the blockchain. Its main function is to calculate the state of the network and execute and compile various types of smart contract. This makes it possible for smart contracts deployed on EVM-compatible networks, such as Binance or Polygon, to be recognized by Ethereum nodes, and allows developers to migrate their Ethereum applications to other EVM-compatible networks easily.

Polygon [30] is a blockchain network that enables connectivity and scalability between different networks. Its main goal is to create a blockchain ecosystem of multiple Ethereum-compatible networks.

Smart contracts [31] specify business logic and agreements between participating nodes in a blockchain network. These agreements are executed automatically when certain conditions are satisfied, dealing with the information transmitted across the network or even extracted from other interconnected systems.

In this work, blockchain technology is used in CEPEDALoCo for storing the complex events detected by a CEP engine in a traceable, secure and immutable way.

2.3. Low code

Low-code [32] is a paradigm which facilitates the development of applications through the use of graphical user interfaces and visual elements. It allow the development of solutions without requiring too much knowledge of technologies or programming languages. In the software development industry, low-code is increasingly present when products are developed without requiring extensive knowledge of a particular technology [33].

Although low-code is a very new term, its fundamentals are not so new. This paradigm can be considered as a more restrictive derivation of Model-Driven Development (MDD) [34], specializing it in certain types of applications such as web or mobile applications [35].

In this context, low-code platforms [36] provide a development environment through graphical user interfaces that are used to create functional applications. This will require additional programming in certain situations. A low-code platform greatly reduces the amount of manual coding required and enables faster application deployment. People with limited programming skills have the opportunity to contribute to the development of large applications. It also reduces configuration, training, deployment, and maintenance costs, among other benefits [37].

Currently, the development of low-code platforms is progressing worldwide. In particular, Node-RED [13] is one of the most popular flow-based visual programming tools. It is based on a low-code paradigm and allows us to connect hardware, APIs, and services through graphical user interface. Moreover, this platform can be used for the development of EDAs. Among other advantages, Node-RED facilitates collaborative application development, allows importing and exporting the created flows and is a lightweight tool, which can be deployed in software containers on most lightweight devices such as Raspberry Pi.

In this work, low-code is used in CEPEDALoCo for the integration of CEP and blockchain, making them more accessible to users who do not have vast knowledge of these technologies. Additionally, it enables the use of multiple IoT devices regardless of their manufacturer or data format.

Table 1
Comparison of proposals related to the contributions of this paper.

Approach	C1	C2	C3	C4
Alamri's work [39]	+	–	+	–
Boubeta-Puig's work [7]	–	+	+	–
Fournier et al. [40]	–	+	+	–
Leal's work [41]	+	+/–	+	–
Settlement [42]	–	–	+	–
Unibright [43]	–	–	+	–
Creatorchain [44]	–	–	+	–
Aurachain [45]	+/–	–	+	–
CEPEDALoCo	+	+	+	+

3. Related work

This section discusses existing proposals with respect to the contributions of this work mentioned in Section 1: C1 (facilitating the integration of multiple IoT devices regardless of data format and manufacturer), C2 (analyzing and correlating real-time streams of data to early detect complex events and reacting quickly to them by triggering a series of actions), C3 (storing complex events ensuring security, immutability, transparency, decentralization and traceability) and C4 (developing EDAs integrating IoT, CEP and blockchain in a user-friendly way through low-code paradigm).

More specifically, each of related proposals is compared in terms of the contributions provided by CEPEDALoCo as follows: – not addressed, +/- partially addressed, or + addressed, as is summarized in Table 1.

Although there are several existing IoT platforms empowered by some of the strongest companies in the world [38], to the best of our knowledge, none of them provides a solution similar to the one presented by this work where IoT is integrated with CEP and blockchain technologies for facilitating the analysis and correlation of real-time data with the aim of detecting complex events that can be stored automatically in an immutable and transparent way. In addition, the presented open-source proposal is flexible since new software components can be developed and integrated into the architecture.

Related works grouped by the mentioned contributions will be discussed below.

3.1. Facilitating the integration of multiple IoT devices

CEPEDALoCo allows the integration of multiple IoT devices regardless of their manufacturer or data format. Thanks to this, multiple heterogeneous devices can be integrated under a single solution to meet user needs. In this context, there are other works that also provide this C1 contribution.

Alamri et al. [39] proposes a prototype using a low-code approach coupled with blockchain technology for personal health records. They aim to keep patient data up to date through the use of IoT devices. In addition, they facilitate access to data as well as secure and private access through the use of blockchain and smart contracts.

Leal et al. [41] present an IoT-based inviolable presence registration service. This implementation relies on blockchain technology to store and retrieve data while ensuring data integrity. Moreover, they use CEP to efficiently record data on the blockchain.

In addition, there is another work that partially provides this contribution. Aurachain [45] is a tool focused to be used by citizens and developers since it has a visual module with drag and drop functions. In particular, it enables rapid development of blockchain applications that can be deployed on Hyperledger or Ethereum networks. It also allows integration with other external applications and services.

3.2. Analyzing and correlating real-time streams of data

CEPEDALoCo also allows to analyze and correlate data streams in real-time for the early detection of complex events and to react quickly to them by triggering a series of actions. In other words, from all the data received, it uses CEP technology to detect situations of interest based on a series of previously defined event patterns and to be able to react to them quickly. There are other works that also provide this C2 contribution, as detailed below.

Boubeta-Puig et al.'s work [7] proposes and implements the integration of blockchain and CEP technologies through a model-driven approach, called CEPchain. Specifically, it provides a graphical tool for creating models of event patterns and smart contracts that can be automatically transformed into code, as well as deployed in a CEP engine and a blockchain network, respectively. Thanks to the integration of both technologies, CEPchain can automatically invoke functions of a smart contract from decisions made in real time by a CEP engine.

Fournier et al. [40] describes a model-based approach capable of creating blockchain applications that have the ability to execute temporal logic. In addition, they use CEP technology for a case study on the cold chain. Thus, their work enables real-time analysis for the detection of situations such as improper temperatures or counterfeit drugs.

Moreover, the mentioned Leal et al.'s work [41] partially provides this contribution.

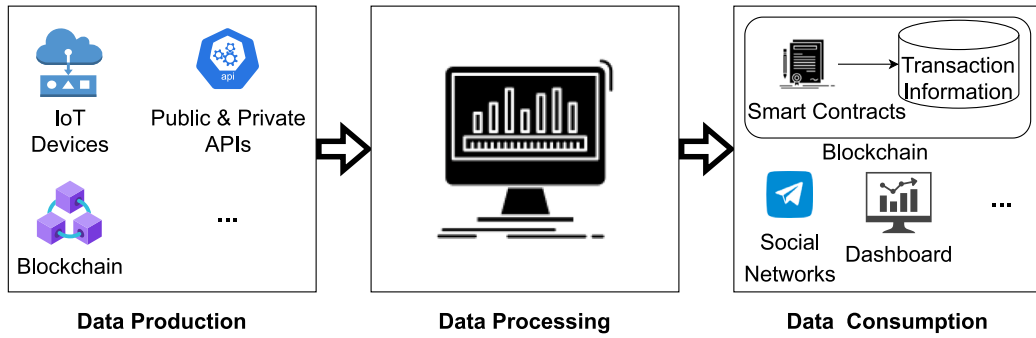


Fig. 1. General overview of the CEPEDALoCo proposal.

3.3. Storing complex events ensuring security, immutability and traceability

CEPEDALoCo allows to store complex events generated in real time in a secure, immutable, transparent, decentralized and traceable way. That is, when a new complex event is generated by the CEP technology, the presented proposal automatically stores each of these complex events in a blockchain network. Thanks to the use of this technology is how this proposal ensures these advantages by the own design of blockchain networks.

All works and tools similar to CEPEDALoCo and shown in Table 1 provide this C3 contribution. The ones that have not yet been mentioned are described below.

Settlemint [42] is a low-code-based Blockchain-Platform-as-a-Service. It allows the creation of blockchain networks from other networks belonging to the EVM such as Ethereum, Polygon or Binance Smart Chain. Thanks to the use of the low-code paradigm, less technical knowledge is required for the creation of this type of solutions. However, this tool is focused on enterprise use due to its high costs.

Unibright [43] is a consulting company that offers low-code based integration tools through a team of blockchain specialists. The goal is to be able to integrate business processes with blockchain without the need for coding. Their framework is designed to help companies integrate blockchain into their processes through their visual tool.

Creatorchain [44] is based on Polkadot, a network that does not belong to the EVM but provides interoperability with those networks such as Ethereum or Polygon. It has a graphical interface for users or developers depending on the purpose of tool use. Moreover, this tool can define, compile and deploy smart contracts graphically.

3.4. Integrating IoT, CEP and blockchain in a user-friendly way

CEPEDALoCo allows the development of EDAs that integrate IoT, CEP and blockchain technologies in a user-friendly way through the low-code paradigm. Therefore, it allows us to make use of very powerful technologies together and without requiring vast technical knowledge in them thanks to the use of the low-code paradigm.

Remarkably, none of the works analyzed and discussed in this section provides this C4 contribution.

4. Proposal

This section presents the proposed architecture, called CEPEDALoCo, that makes the integration of CEP and blockchain possible through the low-code paradigm. This architecture is composed of three main layers (Data Production, Data Processing and Data Consumption) as illustrated in Fig. 1.

The **Data Production** layer in charge of generating information from different sources, such as IoT devices, APIs or blockchain networks.

More specifically, IoT devices can be manufactured by different vendors and measure different parameters, such as temperature, humidity or air quality.

APIs as data sources allow the architecture to communicate with other external products and services without having to know how they are implemented. This can simplify the development of EDAs, saving time and money. APIs provide flexibility, simplify design, administration and usage.

Regarding blockchain data sources, these are composed of several elements such as blocks, transactions and smart contracts. When a smart contract is deployed or one of its functions is invoked, a transaction is created on the blockchain. These transactions are grouped together and new blocks are created. Remarkably, smart contracts, transactions or blocks can be monitored in real time through the presented proposal thanks to the use of software components that have been developed specifically for this purpose.

All the information generated in the Data Production layer will be sent to the next layer, the Data Processing layer.

The **Data Consumption** layer is composed of those devices or services which can consume the data generated by the Data Processing layer such as social networks, dashboards or smart contracts.

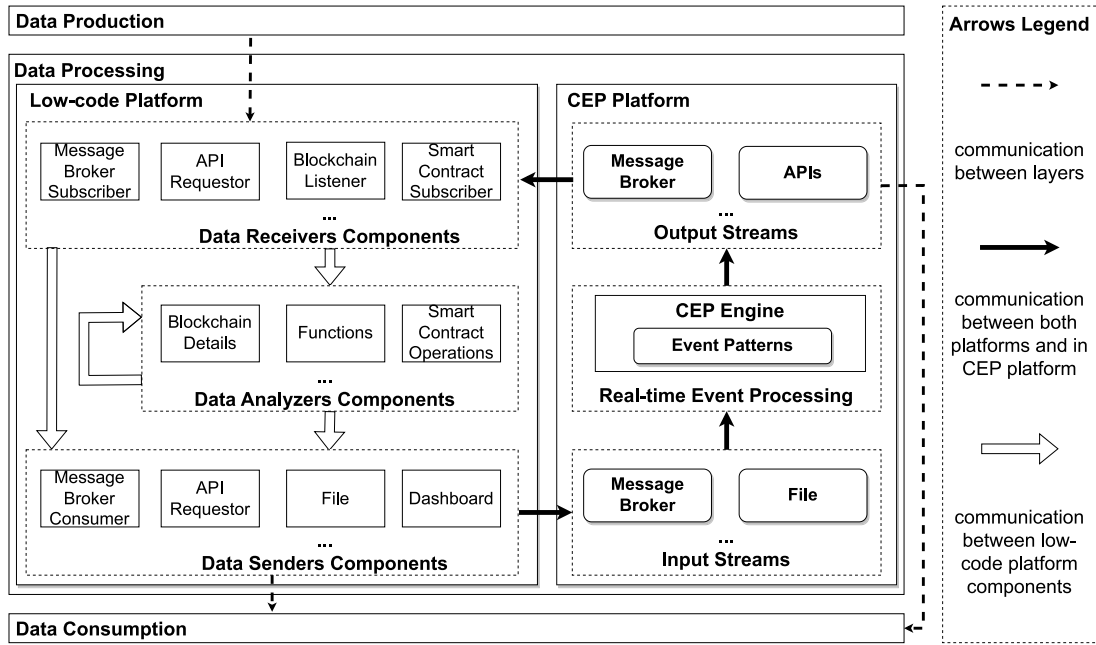


Fig. 2. CEPEDALoCo architecture.

As an example, social networks can be used to disseminate the data and results obtained. A dashboard can be used to represent in a graphical and user-friendly way the conclusions obtained while smart contracts can be used to store data in a secure, reliable, immutable and traceable way.

Data consumers are those elements that allow us to transfer the knowledge generated by the presented proposal to other systems or tools, i.e., they act as possible intermediaries between the presented proposal and other more complex systems.

The **Data Processing** layer is the most important of the whole architecture as it contains the low-code platform and the CEP platform (see Fig. 2). Both platforms can be seen separately as they are deployed in two different software containers. Note that the arrows marked in bold symbolize how information is exchanged between the two platforms as well as within the CEP platform, while the dashed arrows that can be seen in the architecture from the Data Production layer to the Data Processing layer or from the Data Processing layer to the Data Consumption layer symbolize the sending of data between layers.

Specifically, Fig. 2 shows in detail the Data Processing layer of the EDA which is composed of the two platforms previously mentioned and which will be explained in the following subsections.

4.1. Low-code platform

The low-code platform facilitates the development of EDAs. It has a browser-based graphical editor which allows the development of this type of applications in a user-friendly way through flows. These flows are composed of components that can be grouped according to their functionality as follows: (i) data receivers components, which are responsible for obtaining real-time data from the data production layer and introduce it into the low-code platform; (ii) data analyzers components, which are responsible for analyzing and processing the information such as storing it in the blockchain or sending it to the data sending components; and (iii) data senders components, which allow sending all the data from the low-code platform to the CEP platform or to the data consumption layer.

In order to better understand the functioning of the components within the low-code platform, thick arrows are used to represent a set of arrows, i.e., it symbolizes that a data receiver component can send information to multiple data analyzers components and multiple data senders components. Likewise, a data analyzer component can receive information from multiple data receivers components and send information to multiple data analyzers components and multiple data senders components. Finally, the data senders components may receive information from multiple data receivers components and multiple data analyzers components.

Specifically, this proposal is based on an existing low-code platform called Node-RED, which provides an editor that helps users to create flows by dragging and dropping all available graphical nodes on its tool palette. A flow is a directed graph of processing graphical nodes. These graphical nodes are represented by the elements that make up the Node-RED function palette. Also, these elements perform a series of operations and allow the connection of hardware devices, APIs and services, among others. Each node is a piece of code that is implemented in the JavaScript programming language. All available graphical nodes can be easily deployed with a single click.

It should be noted that an extension of the Node-RED tool has been implemented to facilitate the integration of blockchain technology within the low-code paradigm. Each of these developed components has a NodeJS code for all the logic and another HTML code to define the visual aspects. These components can be used to interact with several blockchain networks even at the same time. In this way users will be able to develop EDAs that automatically invoke actions on one or more blockchain networks.

The developed components can be grouped into two groups: (i) smart contract management, and (ii) blockchain monitoring. On the one hand, the smart contract management components improve the accessibility of blockchain technology by supporting the compilation, deployment and interaction operations of smart contracts in different blockchain networks. On the other hand, blockchain monitoring components allow real-time monitoring of the different components previously mentioned in this type of networks, such as smart contracts, transactions and blocks. These components can be combined with each other creating, for example, an EDA which, from a series of values monitored in real time from a blockchain network, triggers a specific action in another blockchain network which will be invoked automatically.

In particular, the main components that have been proposed in the architecture are (more components are available in the low-code platform):

- **Message Broker Subscriber:** this allows to listen in real time for new data received from the data production layer to be fed into the low-code platform. Please note that depending on the communication protocol used, the listening can work in (near) real time.
- **API Requestor (Receiver):** this allows to consume data from external sources in a public or private way to introduce them into the low-code platform.
- **Blockchain Listener:** this allows to obtain information from a blockchain network such as transactions or blocks that are mined in real time.
- **Smart Contract Subscriber:** this allows to subscribe to a smart contract previously deployed in a blockchain network to listen in real time if there is any transaction whose origin or destination is that smart contract.
- **Blockchain Details:** this allows to obtain details of a blockchain network on existing blocks or transactions.
- **Functions:** these allow to define pieces of code in JavaScript language to be able to perform multiple operations with the information that circulates through the low-code platform.
- **Smart Contract Operations:** these allow operations such as compilation, deployment and interaction with smart contracts written in Solidity language.
- **Message Broker Consumer:** this allows to send data from the low-code platform to the CEP platform or to the data consumption layer.
- **API Requestor (Sender):** this allows to make low-code platform data available in an external API (public or private) so that it can be consumed by other applications and services.
- **File:** this allows to save data from the low-code platform to a local or remote host.
- **Dashboard:** this allows to make the data of the low-code platform available to users in a user-friendly way through the use of interactive panels and charts.

4.2. CEP platform

The another key part of the data processing layer refers to the CEP platform which will be in charge of analyzing and correlating the real-time events coming from the low-code platform. Within the CEP platform can be found (i) the input streams, which allow us to obtain the results generated by the low-code platform as input events to the CEP engine, for example, through message brokers; (ii) real-time event processing, where the CEP engine is in charge of correlating all the data received in real time and automatically generating complex events when the conditions previously defined in the event patterns are satisfied; and (iii) output flows, which will be in charge of sending the complex events generated to the low-code platform or to the data consumption layer, for example, through the use of APIs. As shown in Fig. 2, an information flow is created between both platforms which can be repeated iteratively as many times as necessary until the desired result is produced.

Concretely, the main elements that have been proposed in the architecture are (more connectors for input and output streams are available in the CEP platform):

- **Message Broker (Input):** this allows to listen in (near) real time to new data received from the low-code platform in order to enter them into the platform and make them available to the CEP engine.
- **File:** this allows to enter data already existing in the platform and make it available to the CEP engine (not in real time).
- **CEP Engine:** this allows to analyze and correlate all the data received by the CEP platform in real time, detecting complex events by matching a series of previously defined event patterns.
- **Message Broker (Output):** this allows to send complex events detected by the CEP engine to the low-code platform or to the data consumption layer.
- **APIs:** these allow to make available in an external API (public or private) the complex events detected by the CEP engine so that they can be consumed by other applications and services.

Please note that the CEP platform to be used in the architecture can be replaced by other ones. Indeed, it can even be replaced by graphical modeling tools that give support to the modeling and automatic code generation of event patterns such as MEdit4CEP [9]. However, when selecting a CEP platform to work with the architecture, the following minimum requirements should be satisfied:

- It must be able to incorporate data flows through elements such as message brokers or APIs.
- It must allow the definition and deploy of event patterns to analyze and correlate data in real time.
- It must have the ability to send the results of event processing through elements such as message brokers or APIs (optional, but recommended).

5. Case study

To demonstrate the functionality of the CEPEDALoCo proposal, it has been applied to a real-world case study on air quality that makes use of different IoT devices. Air quality is one of the most researched topics in recent years [46].

The hardware requirements of the presented architecture, its development as well as the used IoT devices, the designed low-code flows, the modeled CEP application together with the event patterns, and the blockchain network used in this case study are explained below.

5.1. Hardware requirements

This case study has been deployed on a server with the following characteristics: (i) 2 cores (Intel Xeon E5-2670); (ii) 8 GB memory; and (iii) 25 GB storage.

However, it can be deployed on most lightweight devices such as Raspberry Pi or Orange Pi. The minimum recommended requirements to be taken into account in order to deploy this case study are: (i) 2 cores; (ii) 1 GB memory; and (iii) 5 GB storage.

5.2. Architecture development

For the deployment of the architecture for this case study, Docker [47] software container technology has been used. Specifically, the low-code platform and the CEP platform are located in two different containers hosted on the same server. In addition, other software containers have been used to deploy the message broker for the exchange of messages between both platforms.

Particularly, a series of Netatmo IoT [48] devices have been used as data producers. For the data processing layer, the low-code Node-RED platform [13] has been used, extended with developed components (see Section 4), and the CEP Siddhi Tooling Editor platform [49]. In addition, as a message broker for the exchange of information between the two platforms, Kafka [50] has been chosen as it allows high performance data pipelines. For the data consumption layer, the Polygon [30] blockchain network has been used since it is one of the most used networks with an average of 3 million transactions per day. Please note that the platforms used are a concrete implementation of the proposed architecture, being compatible with other IoT devices, blockchain networks, CEP platforms and services.

5.3. IoT devices

Currently, there are 3 Netatmo smart weather stations [48] distributed in 3 different locations. Two of them are located in two different locations in the city of Cadiz (Spain), and the third one is located in the province of Seville (Spain). These stations are composed of 12 IoT devices capable of measuring temperature, CO₂, humidity, noise, rain, wind and air pollutants, among other things. Specifically, the stations contain the following sensors:

- Smart Weather Station 1 (Cadiz): 1 indoor module, 1 outdoor module, 1 additional indoor module, 1 smart rain gauge and 1 smart anemometer.
- Smart Weather Station 2 (Cadiz): 1 indoor module, 1 outdoor module and 1 smart rain gauge.
- Smart Weather Station 3 (Seville): 1 indoor module, 1 outdoor module and 2 additional indoor module.

Each of these sensors publishes information to the official Netatmo API every 10 min (this frequency is not configurable). This API is consumed by the CEPEDALoCo architecture through the data receivers components which obtain all the information collected by the 3 stations.

More information on the IoT devices used in this case study can be found at: (i) Smart Home Weather Station [51]; (ii) Smart Anemometer [52]; (iii) Smart Rain Gauge [53]; and (iv) Additional Smart Indoor Module [54].

5.4. Low-code flows

To support this case study, 5 node flows have been created on the Node-RED low-code platform. Each of these flows is composed of a series of components and has a different functionality:

- **Station1Flow:** this is the flow in charge of obtaining the information from station 1, splitting the information for each of its modules and sending it to the CEP platform through a message broker to its respective topics.
- **Station2Flow:** this is the flow in charge of obtaining the information from station 2, splitting the information for each of its modules and sending it to the CEP platform through a message broker to its respective topics.
- **Station3Flow:** this is the flow in charge of obtaining the information from station 3, splitting the information for each of its modules and sending it to the CEP platform through a message broker to its respective topics.

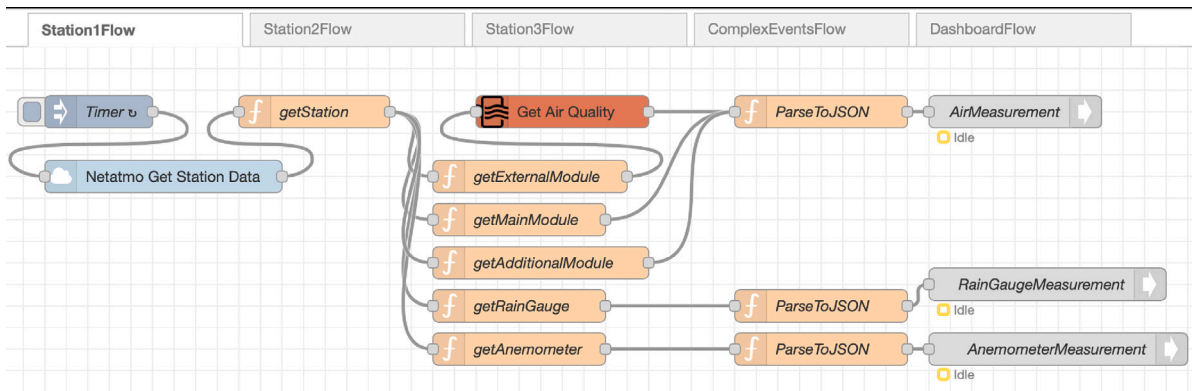


Fig. 3. Graphical editor of the low-code platform with the “Station1Flow”.

- **ComplexEventsFlow:** this is the flow responsible for receiving in real time the complex events generated by the CEP platform through the message broker. These complex events will be automatically registered in the Polygon blockchain network and, at the same time, notified to interested users via Telegram. The result produced by this flow can be seen in Section 6.
- **DashboardFlow:** this is the flow for monitoring in real time the smart contract used for this case study as well as graphically displaying the complex events registered in a blockchain network via a dashboard. Specifically, when a new complex event is received in the blockchain, i.e. when a new transaction has taken place, the number of complex events already registered in the blockchain will be requested and a graph with the statistics of the number of complex events detected and classified by type will be automatically displayed. The result produced by this flow can be seen in Section 6.

As an example, one of the flows mentioned above is explained in detail. Fig. 3 shows the flow used for station 1. Blue nodes are in charge of executing the flow every 10 min and obtaining the information from the IoT devices of station 1. Orange nodes are responsible for making transformations and modifications to the information in order to split it and transform it into the desired format. “Get Air Quality” node is in charge of adding the information on air pollutants and Air Quality Index (AQI) to the IoT device deployed outdoor. Finally, the grey nodes are responsible for sending the information through the Kafka message broker so that the input streams of the CEP platform can receive the data.

Please note that thanks to the use of the low-code paradigm, the presented architecture can be used in multiple domains other than this case study, such as water supply network management, cybersecurity or smart cities.

5.5. CEP application

Fig. 4 shows an extract of the CEP application defined for this case study through the Siddhi Tooling [49], a CEP application development tool which provides two interfaces (code editor and graphical editor) to develop Siddhi applications.

Note that the elements containing an arrow symbolize the input and output streams through which the CEP platform is able to receive data from the low-code platform and send it back. Besides, the elements called “query” plus a number are the defined event patterns, which are based on a set of rules to analyze and correlate the received data to automatically detect situations of interest (complex events) in real time.

Particularly, 25 event patterns have been defined graphically via the Siddhi Tooling. These can be classified into 4 different groups:

- **Malfunction Event Patterns:** These event patterns are intended to detect when there is an error in the data readings from any of the IoT sensors.
- **Outdoor Air Quality Event Patterns:** these event patterns are intended to monitor outdoor air quality. That is, detecting when the AQI may represent a health risk. Please note that the AQI level has been calculated with reference to the values in Table 2.
- **Indoor Air Quality Event Patterns:** these event patterns are intended to monitor indoor air quality. In this case, these measure factors such as temperature, humidity or CO₂.
- **External Factors Event Patterns:** these event patterns are intended to monitor other external factors that may represent a risk, such as wind, rain or noise.

As an example, Listing 1 shows the basic structure of a CEP application: data sources, event patterns and data sinks. More specifically, Line 1 defines the name of the CEP app; Lines 3 to 7 define the data source (in this case, a Kafka message broker) with its configuration options; Line 8 defines the data stream coming from the message broker that corresponds to the data obtained in real time by the IoT devices; Lines 10 to 14 define a data sink (in this case, a Kafka message broker) with its configuration options;

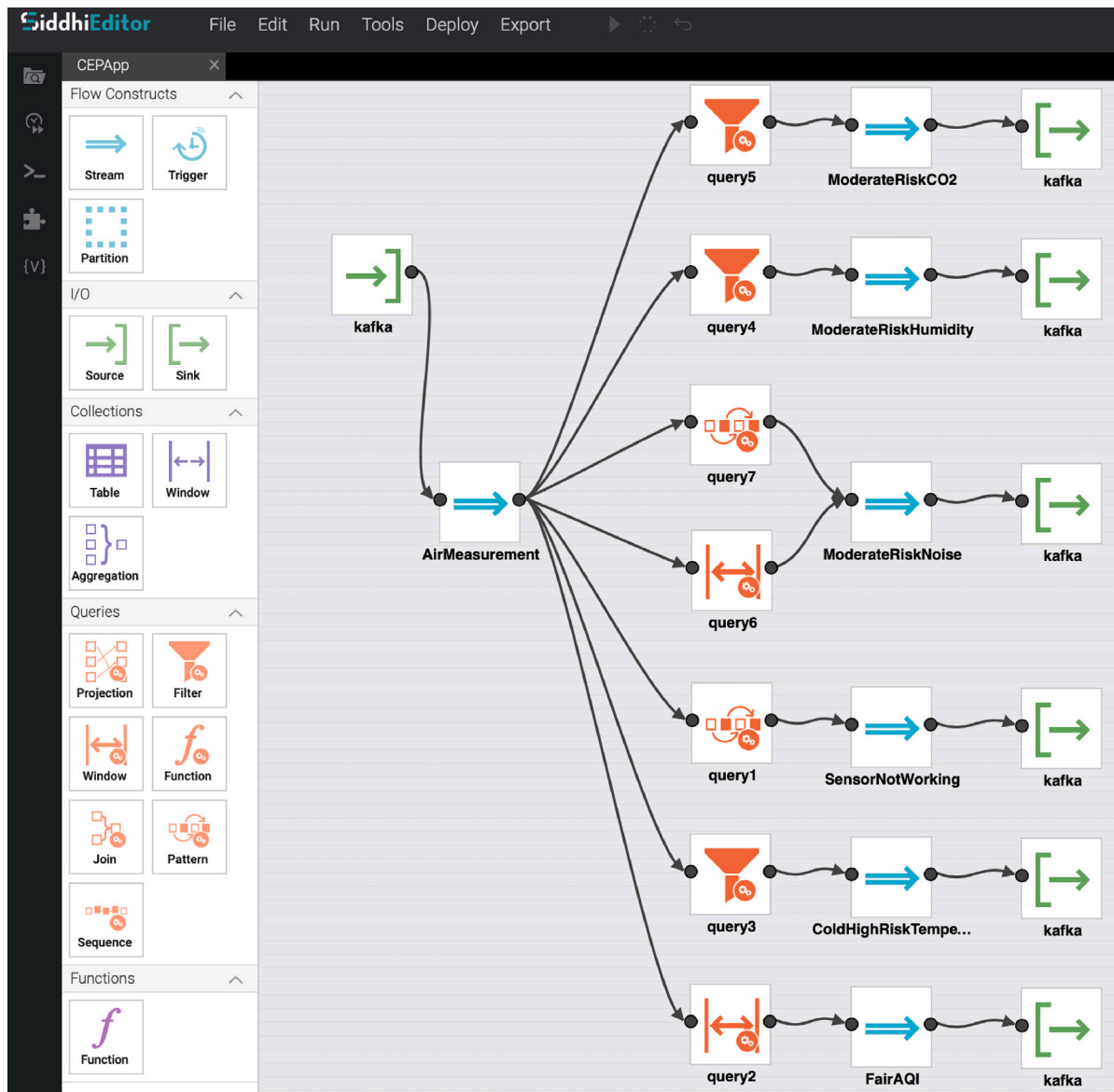


Fig. 4. Excerpt from the design view of the CEP application.

Table 2

AQI categories.

Source: Extracted from [55].

Air quality	Pollutants			
Name (Level)	NO ₂ (µg/m ³)	PM ₁₀ (µg/m ³)	O ₃ (µg/m ³)	PM _{2.5} (µg/m ³)
Good (1)	0–50	0–25	0–60	0–15
Fair (2)	50–100	25–50	60–120	15–30
Moderate (3)	100–200	50–90	120–180	30–55
Poor (4)	200–400	90–180	180–240	55–110
Very Poor (5)	>400	>180	>240	>110

Table 3
Event patterns defined through the Siddhi tooling editor.

Event pattern name	Conditions
SensorNotWorking (temperature)	2 equal measures
SensorNotWorking (rain gauge)	2 equal measures
SensorNotWorking (anemometer)	2 equal measures
FairAQI	Level 2
ModerateAQI	Level 3
PoorAQI	Level 4
VeryPoorAQI	Level 5
ColdHighRiskTemperature	<13 °C
ColdRiskTemperature	≥13 °C & <16 °C
HotRiskTemperature	≥32 °C & <40 °C
HotMediumRiskTemperature	≥40 °C & <54 °C
HotHighRiskTemperature	≥54 °C
ModerateRiskHumidity	<30% & >70%
ModerateRiskCO2	>1000 ppm & <2000 ppm
MediumRiskCO2	≥2000 ppm & <5000 ppm
HighRiskCO2	>5000 ppm
ModerateWind	≥21 km/h & <41 km/h
StrongWind	≥41 km/h & <71 km/h
VeryStrongWind	≥71 km/h & <120 km/h
HurricaneWind	≥120 km/h
StrongRain	≥15 mm/h & <30 mm/h
VeryStrongRain	≥30 mm/h & <60 mm/h
TorrentialRain	≥60 mm/h
ModerateRiskNoise	≥85 Db (for 8 h)
ModerateRiskNoise	≥100 Db (for 15 min)

then, Lines 16 to 22 show the *ModerateRiskNoise* event pattern that makes use of the previously defined data source and sink. This pattern will occur when an average noise equal to or greater than 85 decibels is detected in an 8-hour time window. Note that the complete implementation code of this CEP application including all data sources, event patterns and data sinks are available to be downloaded from [56].

Listing 1: Excerpt of the CEP application code used in the case study.

```

1  @App:name ("CEPApp")
2
3  @source(type='kafka', topic.list='AirMeasurement',
4    threading.option='single.thread', group.id="group1",
5    bootstrap.servers='${KAFKA_ADDRESS}',
6    @map(type='json', fail.on.missing.attribute="false",
7    @attributes(...)))
8  define stream AirMeasurement(...);
9
10 @sink(type='kafka', bootstrap.servers='${KAFKA_ADDRESS}',
11   topic='ModerateRiskNoise', @map(type='json'))
12 define stream ModerateRiskNoise(timestamp string,
13   sensorId string, longitude double, latitude double,
14   sensorTimestamp string, avgNoise double);
15
16 from AirMeasurement#window.time(8 hour)
17 select time:currentTimestamp('CET') as timestamp,
18   sensorId, longitude, latitude, sensorTimestamp,
19   avg(noise) as avgNoise
20 group by sensorId
21 having avgNoise >= 85
22 insert into ModerateRiskNoise;
```

In order to establish which thresholds should be defined in the event patterns according to the factors being measured, it was previously investigated which values could be harmful to health as well as their consequences. As a result, Table 3 shows the thresholds used in this case study for each of the factors measured.

5.6. Blockchain network

As previously mentioned, all complex events produced by the CEP platform are automatically registered in the Polygon blockchain network. To carry out this registration, a smart contract called *ComplexEvents* has been implemented using the Solidity

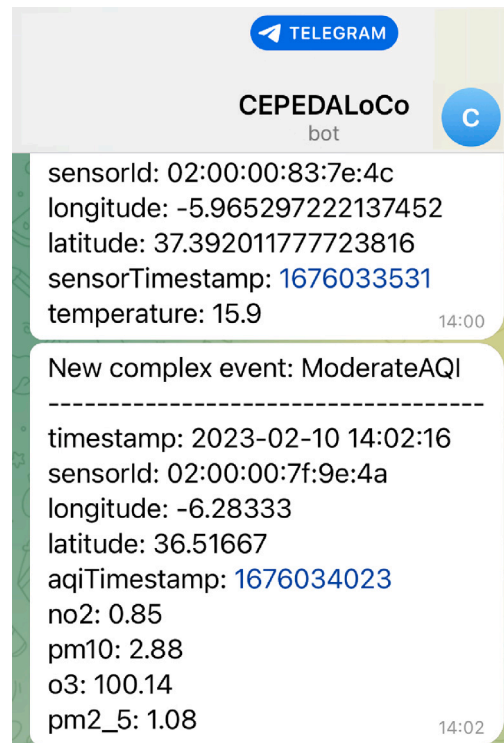


Fig. 5. Example of a new complex event notification in Telegram.

language to define the necessary logic to be able to automatically register new complex events and obtain all the ones that have already been registered. The information that is obtained from the CEP platform and, therefore, is registered in the blockchain is:

- **timestamp:** the timestamp value when the CEP platform has generated the complex event.
- **name:** the name of the type of complex event that has been generated by the CEP platform.
- **content:** the information contained in the generated complex event such as sensor id, temperature, longitude and latitude.

Every time a new complex event is registered in the blockchain network, a transaction takes place in a transparent and immutable way. Therefore, any user can query the details of these transactions from anywhere in the world and at any time.

Please note that due to space restrictions, all the screenshots of the case study as well as the results obtained are available at [56].

6. Results and evaluation

This section evaluates the results of applying the CEPEDALoCo proposal to the case study described in Section 5. In addition, this proposal has been tested by a group of independent users whose results are also discussed in this section.

6.1. Results

Such a case study was run for 15 days (from 25/01/2023 to 08/02/2023). During the entire test run period, the IoT devices worked correctly and uninterruptedly. Moreover, during its operation, the complex events detected in real time were automatically stored in the Polygon blockchain. This was possible thanks to the use of the developed smart contract, ComplexEvents (see Section 5.6), which was deployed in that blockchain network and can be queried by anyone via the following public address [57]: [0xfaeA61bd67994c24d3Ac2458a79ce794c8376021](#)

By using CEPEDALoCo, which integrates CEP and blockchain technologies, all these complex events were automatically registered and notified by Telegram (see Fig. 5) through the developed low-code flows. Additionally, the numbers of complex events registered in the blockchain was graphically displayed through a dashboard (see Fig. 6). In this figure, it can be checked that the major number of detected patterns were related to cold temperatures, humidity and CO2 problems. With respect to AQI, it is noted that, as a general rule, there were no risks associated with this index. Additionally, there were some occasional errors in the reading of data from some sensors. Therefore, it can be affirmed that the EDA presented in Section 4 works correctly.

It is important to highlight that the registration of information in a blockchain network involves a monetary cost. In this context, the number of complex events registered in the blockchain network in the form of transactions is 11,229. The cost of storing all

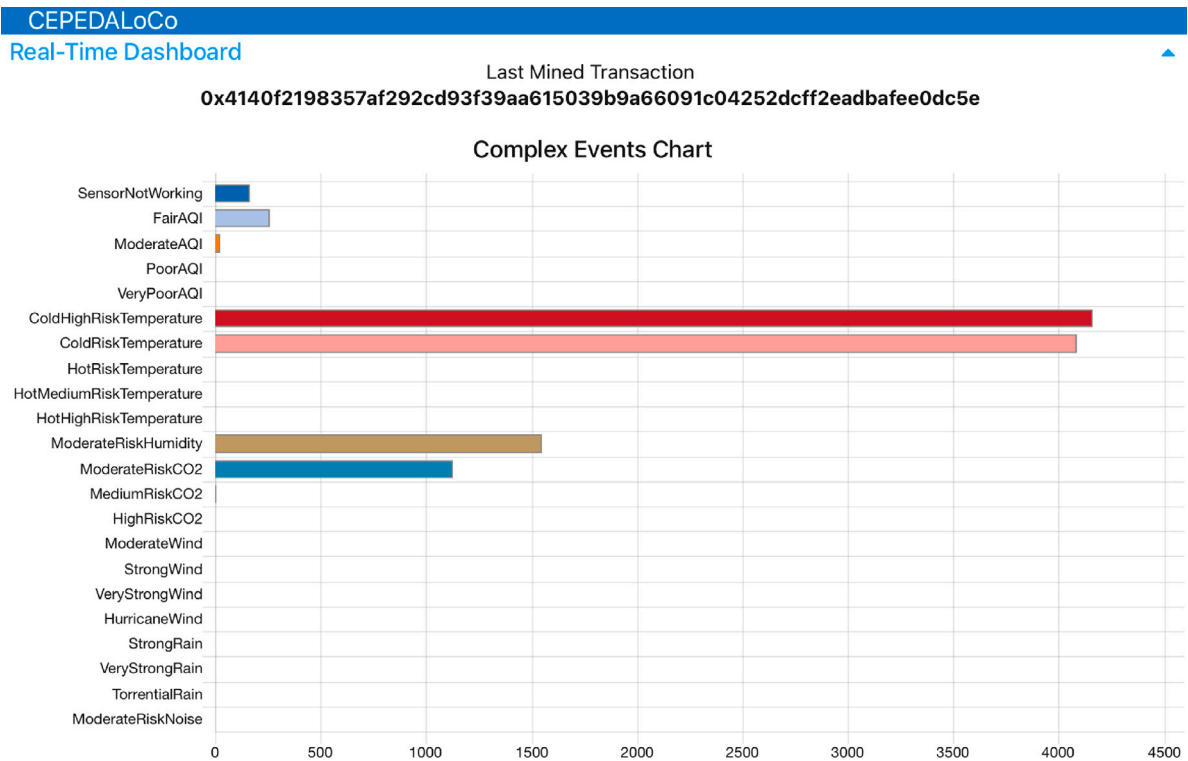


Fig. 6. Dashboard with the number of complex events registered in the blockchain.

these transactions has been \$11.15, i.e. the cost of storing each of these complex events in the blockchain network has been less than \$0.001. Therefore, the presented proposal is feasible in economic terms due to its reduced cost. Please note that in this case study a very high number of transactions has been handled for testing purposes, in other circumstances it would not be expected to have 11,229 transactions in 15 days.

6.2. Evaluation

It has been considered essential to know the opinion of users regarding the functionalities provided by the presented proposal. To this end, a questionnaire has been carried out and is available in PDF version at [56], which contains questions on personal information and technological competence, and functionality of the application.

This questionnaire was conducted by a group of students belonging to a Master's Degree Program in Software Engineering at the Western Norway University of Applied Sciences, Bergen (Norway). Specifically, a group of 6 students between 18 and 45 years old. After a brief explanation of the proposal, they were challenged to develop a task. This task consisted of the graphical design of an EDA that is capable of connecting the IoT devices presented in the case study (see Section 5) to the Polygon blockchain network through the CEPEDALoCo architecture. After this, they had to complete the questionnaire for evaluating the tool. Please note that the number of respondents is not a representative sample, however, it has allowed us to obtain information of interest for the future of the proposal.

The results of the questionnaire conducted on the functionality evaluation of the tool are available at [56]. These can be summarized as follows:

- 100% of the respondents think that the functionalities offered by the tool are useful.
- 100% of the respondents think that there are no irrelevant user interface elements or functionalities.
- 100% of the respondents think that the tool should be used by users with programming skills.
- 100% of the respondents think that the tool allows to reduce to a greater or lesser extent the time needed to create new software solutions using technologies such as blockchain.
- The tool meets user expectations by 86.67% (average value).

Therefore, the results demonstrate that the proposed architecture is capable of enabling the development of user-friendly EDAs that integrate multiple IoT devices with the real-time data analysis and correlation by the CEP engine and the automatic storage of complex events in a secure and transparent way through the low-code paradigm.

7. Conclusions and future work

In this paper, an EDA for integrating CEP and blockchain through the low-code paradigm, CEPEDALoCo, has been proposed. This architecture enables the integration of multiple IoT devices, regardless of their data format or manufacturer, with blockchain technology. In addition, it allows software developers that are non-experts in blockchain to create EDAs that automatically trigger actions on a blockchain network. It also allows to analyze and process large amounts of real-time data from multiple data sources for complex event detection.

CEPEDALoCo is based on a low-code platform that facilitates the development of this kind of applications in a faster way generating value in less time. Among other benefits, it provides better code quality, greater flexibility, task automation, component reuse and integration with other external systems, applications and services.

CEPEDALoCo provides a graphical editor as a result of the extension of the Node-RED low-code tool through the development of new software components, which make the use of blockchain technology within the platform possible. Additionally, it provides multiple components to exchange information with the CEP platform, such as message brokers or APIs.

Although CEPEDALoCo can be applied to multiple domains, in this paper it has been applied to a case study on air quality to demonstrate the validity of the proposal. Specifically, 12 different IoT devices which measure several factors such as temperature, humidity or CO₂ levels have been used.

Moreover, the Siddhi Tooling Editor has been used to graphically define 25 event patterns that allow for the automatic detection of complex events on indoor and outdoor air quality as well as other factors. Through the designed EDA, complex events detected by the CEP engine have been automatically stored in the Polygon blockchain network, ensuring security, immutability, transparency and traceability of these events.

Finally, the results obtained from running the case study for 15 days were evaluated, focusing on aspects such as the number of complex events detected or costs related to their storage in the blockchain network. These results show that the presented proposal works as expected and is feasible in economic terms due to its reduced costs. Besides, CEPEDALoCo has been tested by an independent group of users in order to evaluate and demonstrate its functionality.

As future work, it is planned to reduce the complexity of the architecture by integrating the CEP engine into the low-code platform. In this way, it will not be necessary to have two different platforms on which to exchange data, but rather, everything will be integrated in the graphical editor and the data will be able to circulate under the same node flow without the need to exchange information with any external platform, as this is currently the case with CEP. It is also planned to conduct usability studies with a representative group of bachelor and master students enrolled on subjects in software engineering through the use of standards such as System Usability Scale (SUS). Furthermore, the developed architecture will be applied to other real-world IoT domains, such as water management supply and cybersecurity networks. Finally, it is intended to perform scalability and performance tests on the presented architecture in order to assess its limitations and possible improvements to be made in these terms.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Research data and code are available at the following Mendeley Data Repository: <https://doi.org/10.17632/ngshc6y5kv.1>.

Acknowledgments

This work was partly supported by grant PID2021-122215NB-C33 (AwESOMe) funded by MCIN/AEI/10.13039/501100011033/ and ERDF “A way to do Europe”, by grant P20_00865 (DECISION) funded by the Andalusian Plan for Research, Development and Innovation and ERDF, and by grant EST2023-011 funded by the Research Plan from the University of Cadiz 2022–2023, Spain. We would like to thank Javier Berrocal for recommending the purchase of Netatmo sensors. Jesús Rosa-Bilbao would also like to thank the Department of Computer Science, Electrical Engineering and Mathematical Sciences for their hospitality when visiting them at Western Norway University of Applied Sciences, Bergen, Norway, where part of this work was developed.

References

- [1] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, M. Ayyash, Internet of things: A survey on enabling technologies, protocols, and applications, *IEEE Commun. Surv. Tutor.* 17 (4) (2015) 2347–2376, <http://dx.doi.org/10.1109/COMST.2015.2444095>.
- [2] Knud Lasse Lueth, State of the IoT 2018: Number of IoT devices now at 7B – market accelerating, 2023, <https://iot-analytics.com/state-of-the-iot-update-q1-q2-2018-number-of-iot-devices-now-7b/> (accessed 02 February 2023).
- [3] L. Hughes, Y.K. Dwivedi, N.P. Rana, M.D. Williams, V. Raghavan, Perspectives on the future of manufacturing within the industry 4.0 era, *Prod. Plan. Control* 33 (2–3) (2022) 138–158, <http://dx.doi.org/10.1080/09537287.2020.1810762>.
- [4] M. Hosseini Shirvani, M. Masdari, A survey study on trust-based security in internet of things: Challenges and issues, *Internet Things* 21 (2023) 100640, <http://dx.doi.org/10.1016/j.iot.2022.100640>.

- [5] D. Corral-Plaza, G. Ortiz, I. Medina-Bulo, J. Boubeta-Puig, MEdit4CEP-SP: A model-driven solution to improve decision-making through user-friendly management and real-time processing of heterogeneous data streams, *Knowl.-Based Syst.* 213 (2021) 106682, <http://dx.doi.org/10.1016/j.knosys.2020.106682>.
- [6] J. Rosa-Bilbao, J. Boubeta-Puig, A. Rutle, EDALoCo: Enhancing the accessibility of blockchains through a low-code approach to the development of event-driven applications for smart contract management, *Comput. Stand. Interfaces* 84 (2023) 103676, <http://dx.doi.org/10.1016/j.csi.2022.103676>.
- [7] J. Boubeta-Puig, J. Rosa-Bilbao, J. Mendling, CEPchain: A graphical model-driven solution for integrating complex event processing and blockchain, *Expert Syst. Appl.* 184 (Article 115578) (2021) <http://dx.doi.org/10.1016/j.eswa.2021.115578>.
- [8] G. Ortiz, I. Castillo, A. García-de Prado, J. Boubeta-Puig, Evaluating a flow-based programming approach as an alternative for developing CEP applications in IoT, *IEEE Internet Things J.* 9 (13) (2022) 11489–11499, <http://dx.doi.org/10.1109/JIOT.2021.3130498>.
- [9] J. Boubeta-Puig, G. Ortiz, I. Medina-Bulo, MEdit4CEP: A model-driven solution for real-time decision making in SOA 2.0, *Knowl.-Based Syst.* 89 (2015) 97–112, <http://dx.doi.org/10.1016/j.knosys.2015.06.021>.
- [10] X. Xu, I. Weber, M. Staples, *Architecture for Blockchain Applications*, Springer International Publishing, Cham, 2019, <http://dx.doi.org/10.1007/978-3-030-03035-3>.
- [11] T. Domínguez-Bolaño, O. Campos, V. Barral, C.J. Escudero, J.A. García-Naya, An overview of IoT architectures, technologies, and existing open-source projects, *Internet Things* 20 (2022) 100626, <http://dx.doi.org/10.1016/j.iot.2022.100626>.
- [12] L. Almonte, I. Cantador, E. Guerra, J. de Lara, Towards automating the construction of recommender systems for low-code development platforms, in: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, MODELS '20*, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1–10, <http://dx.doi.org/10.1145/3417990.3420200>.
- [13] OpenJS Foundation, Node-RED, 2023, <https://nodered.org/> (accessed 2 February 2023).
- [14] O. Etzion, P. Niblett, *Event Processing in Action*, first ed., Manning Publications Co., Greenwich, CT, USA, 2010.
- [15] J. Boubeta-Puig, G. Ortiz, I. Medina-Bulo, A model-driven approach for facilitating user-friendly design of complex event patterns, *Expert Syst. Appl.* 41 (2) (2014) 445–456, <http://dx.doi.org/10.1016/j.eswa.2013.07.070>.
- [16] M. Castillo-Cara, G. Mondragón-Ruiz, E. Huaranga-Junco, E. Arias Antúnez, L. Orozco-Barbosa, SAVIA: Smart city citizen security application based on fog computing architecture, *IEEE Lat. Am. Trans.* 17 (07) (2019) 1171–1179, <http://dx.doi.org/10.1109/TLA.2019.8931206>.
- [17] A. Tenorio-Trigoso, M. Castillo-Cara, G. Mondragón-Ruiz, C. Carrión, B. Caminero, An analysis of computational resources of event-driven streaming data flow for internet of things: A case study, *Comput. J.* 66 (1) (2021) 47–60, <http://dx.doi.org/10.1093/comjnl/bxab143>.
- [18] G. Mondragón-Ruiz, A. Tenorio-Trigoso, M. Castillo-Cara, M.B. Caminero, C. Carrión, An experimental study of fog and cloud computing in CEP-based real-time IoT applications, *J. Cloud Comput.* 10 (1) (2021) 32, <http://dx.doi.org/10.1186/s13677-021-00245-7>.
- [19] M. Ananthanarayanan, R. Mishra, V. Chakka, How integrated process management completes the blockchain jigsaw, *Blockchain* (2018) 4–7.
- [20] S. Shukla, S. Thakur, S. Hussain, J.G. Breslin, S.M. Jameel, Identification and authentication in healthcare internet-of-things using integrated fog computing based blockchain model, *Internet Things* 15 (2021) 100422, <http://dx.doi.org/10.1016/j.iot.2021.100422>.
- [21] G. Betarte, J.D. Campo, A. Delgado, L. González, Á. Martín, R. Martínez, B. Muracciole, Contact tracing solutions for COVID-19: applications, data privacy and security, *CLEI Electron. J.* 25 (2) (2022) <http://dx.doi.org/10.19153/cleiej.25.2.4>.
- [22] Y. Yu, Y. Zhao, Y. Li, X. Du, L. Wang, M. Guizani, Blockchain-based anonymous authentication with selective revocation for smart industrial applications, *IEEE Trans. Ind. Inform.* 16 (5) (2020) 3290–3300, <http://dx.doi.org/10.1109/TII.2019.2944678>.
- [23] D. Puthal, N. Malik, S.P. Mohanty, E. Kougiannos, C. Yang, 1, *IEEE Consum. Electron. Mag.* 7 (2) (2018) 18–21, <http://dx.doi.org/10.1109/MCE.2017.2776459>.
- [24] D. Bruneo, S. Distefano, M. Giacobbe, A. Longo Minnola, F. Longo, G. Merlino, D. Mulfari, A. Panarello, G. Patanè, A. Puliafito, C. Puliafito, N. Tapas, An IoT service ecosystem for smart cities: The #smartME project, *Internet Things* 5 (2019) 12–33, <http://dx.doi.org/10.1016/j.iot.2018.11.004>.
- [25] Q. Wang, X. Zhu, Y. Ni, L. Gu, H. Zhu, Blockchain for the IoT and industrial IoT: A review, *Internet Things* 10 (2020) 100081, <http://dx.doi.org/10.1016/j.iot.2019.100081>, Special Issue of the Elsevier IoT Journal on Blockchain Applications in IoT Environments.
- [26] D.S.W. Ting, L. Carin, V. Dzau, T.Y. Wong, Digital technology and COVID-19, *Nat. Med.* 26 (4) (2020) 459–461, <http://dx.doi.org/10.1038/s41591-020-0824-5>.
- [27] J. Rosa-Bilbao, J. Boubeta-Puig, RectorDApp: Decentralized application for managing university rector elections, in: *2021 IEEE International Conference on Service-Oriented System Engineering, SOSE, IEEE, Oxford, United Kingdom, 2021*, pp. 161–165, <http://dx.doi.org/10.1109/SOSE52839.2021.00024>.
- [28] D. Drescher, *Blockchain Basics: A Non-Technical Introduction in 25 Steps*, first ed., A Press, 2017.
- [29] A. Busse, J. Eberhardt, S. Tai, EVM-perf: High-precision EVM performance analysis, in: *2021 IEEE International Conference on Blockchain and Cryptocurrency, ICBC, 2021*, pp. 1–8, <http://dx.doi.org/10.1109/ICBC51069.2021.9461058>.
- [30] Polygon technology, Polygon, 2023, <https://polygon.technology> (accessed 11 February 2023).
- [31] Á.J. Varela-Vaca, A.M. Reina Quintero, Smart contract languages: A multivocal mapping study, *ACM Comput. Surv.* 54 (1) (2021) <http://dx.doi.org/10.1145/3423166>.
- [32] C. Ploder, R. Bernsteiner, S. Schlögl, C. Gschliesser, The future use of LowCode/NoCode platforms by knowledge workers – an acceptance study, in: L. Uden, I.-H. Ting, J.M. Corchado (Eds.), *Knowledge Management in Organizations*, Springer International Publishing, Cham, 2019, pp. 445–454.
- [33] L. Brunschwig, R. Campos-López, E. Guerra, J. de Lara, Towards domain-specific modelling environments based on augmented reality, in: *IEEE/ACM 43rd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2021, pp. 56–60, <http://dx.doi.org/10.1109/ICSE-NIER52604.2021.00020>.
- [34] J. Cabot, Positioning of the low-code movement within the field of model-driven engineering, in: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, MODELS '20*, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1–3, <http://dx.doi.org/10.1145/3417990.3420210>.
- [35] L.N. Sánchez-Morales, G. Alor-Hernández, V.Y. Rosales-Morales, C.A. Cortes-Camarillo, J.L. Sánchez-Cervantes, Generating educational mobile applications using UUIDs identified by artificial intelligence techniques, *Comput. Stand. Interfaces* 70 (2020) 103407, <http://dx.doi.org/10.1016/j.csi.2019.103407>.
- [36] S. Farshidi, S. Jansen, S. Fortuin, Model-driven development platform selection: four industry case studies, *Softw. Syst. Model.* (2021) <http://dx.doi.org/10.1007/s10270-020-00855-w>.
- [37] B. Kenneweg, I. Kasam, M. McMullen, *Building Low-Code Applications with Mendix: Discover Best Practices and Expert Techniques to Simplify Enterprise Web Development*, Vol. 20, Packt Publishing, 2021, pp. 1525–1551.
- [38] Gartner, Global industrial IoT platforms reviews and ratings, 2023, <https://www.gartner.com/reviews/market/global-industrial-iot-platforms> (accessed 06 February 2023).
- [39] B. Alamri, I.T. Javed, T. Margaria, A GDPR-compliant framework for IoT-based personal health records using blockchain, in: *2021 11th IFIP International Conference on New Technologies, Mobility and Security, NTMS, 2021*, pp. 1–5, <http://dx.doi.org/10.1109/NTMS49979.2021.9432661>.
- [40] F. Fournier, I. Skarbovsky, Enriching smart contracts with temporal aspects, in: J. Joshi, S. Nepal, Q. Zhang, L.-J. Zhang (Eds.), *Blockchain – ICBC 2019*, in: *Lecture Notes in Computer Science*, Springer International Publishing, Cham, 2019, pp. 126–141, http://dx.doi.org/10.1007/978-3-030-23404-1_9.
- [41] M. Leal, F. Pisani, M. Endler, A blockchain-based service for inviolable presence registration of mobile entities, *J. Braz. Comput. Soc.* 27 (2021) <http://dx.doi.org/10.1186/s13173-021-00104-y>.
- [42] SettleMint, SettleMint, 2023, <https://www.settlemint.com> (accessed 2 February 2023).

- [43] Unibright IT GmbH, Unibright, 2023, <https://unibright.io> (accessed 2 February 2023).
- [44] Creator Platform, Creator, 2023, <https://www.creatorchain.network> (accessed 2 February 2023).
- [45] Aurachain, Aurachain, 2023, <https://aurachain.ch> (accessed 2 February 2023).
- [46] M. Méndez, M.G. Merayo, M. Núñez, Machine learning algorithms to forecast air quality: a survey, *Artif. Intell. Rev.* (2023) 1–36, <http://dx.doi.org/10.1007/s10462-023-10424-4>.
- [47] Docker, Docker, 2023, <https://www.docker.com/> (accessed 13 April 2023).
- [48] Netatmo, Smart home weather station, 2023, <https://www.netatmo.com/en-gb/weather/weatherstation> (accessed 11 February 2023).
- [49] Siddhi, Siddhi, 2023, <https://siddhi.io/en/v5.1/docs/tooling/> (accessed 25 January 2023).
- [50] Apache Kafka, Kafka, 2023, <https://kafka.apache.org> (accessed 13 April 2023).
- [51] Netatmo, Smart home weather station, 2023, <https://www.netatmo.com/smart-weather-station> (accessed 13 April 2023).
- [52] Netatmo, Smart anemometer, 2023, <https://www.netatmo.com/smart-anemometer> (accessed 13 April 2023).
- [53] Netatmo, Smart rain gauge, 2023, <https://www.netatmo.com/smart-rain-gauge> (accessed 13 April 2023).
- [54] Netatmo, Additional smart indoor module, 2023, <https://www.netatmo.com/additional-smart-indoor-module> (accessed 13 April 2023).
- [55] OpenWeather, OpenWeather, 2023, <https://openweathermap.org/api/air-pollution> (accessed 25 January 2023).
- [56] J. Rosa-Bilbao, J. Boubeta-Puig, A. Rutle, Dataset for CEPEDALoCo: An event-driven architecture for integrating complex event processing and blockchain through low-code, 2023, Mendeley Data, v1, <https://doi.org/10.17632/ngshc6y5kv.1> (accessed 18 April 2023).
- [57] Polygon Scan Mumbai, Complex events smart contract transactions, 2023, <https://mumbai.polygonscan.com/address/0xfaeA61bd67994c24d3Ac2458a79ce794c8376021> (accessed 17 February 2023).



Jesús Rosa-Bilbao is a Ph.D. student in the UCASE Software Engineering Research Group at the University of Cadiz (UCA), Spain. He received his B.Sc. degree in Computer Science specialized in Software Engineering from UCA in 2019. In addition, he received his M.Sc. degree in Cybersecurity from UCA, his M.Sc. degree in Project Management from the European Business School in Barcelona (ENEB) and his M.Sc. degree in Business Administration from ENEB in 2020. He also received his M.Sc. degree in Big Data and Business Intelligence from ENEB in 2021. His research interests include complex event processing, event-driven service-oriented architecture, business process modeling, blockchain and cybersecurity.



Juan Boubeta-Puig received the Ph.D. degree in Computer Science and Engineering from the University of Cadiz (UCA), Cádiz, Spain, in 2014. He is an Associate Professor with the Department of Computer Science and Engineering, UCA. His research interests include real-time big data analytics through complex event processing, event-driven service-oriented architecture, Internet of things, blockchain and model-driven development of advanced user interfaces, and their application to smart cities, industry 4.0, e-health, and cybersecurity. Dr. Boubeta-Puig was honored with the Extraordinary Ph.D. Award from UCA and the Best Ph.D. Thesis Award from the Spanish Society of Software Engineering and Software Development Technologies.



Adrian Rutle holds a Ph.D. in Computer Science from the University of Bergen, Norway. Rutle is a professor at the Department of Computer science, Electrical engineering and Mathematical sciences at the Western Norway University of Applied Sciences (HVL), Bergen. Rutle's main interest is applying theoretical results from the field of model-driven software engineering to practical domains and has expertise in the development of modeling frameworks and domain-specific modeling languages. He also conducts research in the fields of modeling and simulation for robotics, eHealth, digital fabrication, smart systems and machine learning.